

Documentación del módulo servidor de eShop Core

Instituto Superior de Formación Técnica N. º 179 Dr. Carlos Pellegrini

Carrera: Tecnicatura Superior en Análisis de Sistemas

Materia: Algoritmos y Estructura de Datos III

Docente: Hansen Andres

Ayudante de Catedra: Rios Emmanuel

Tema: Desarrollo de aplicación con TAD

Estudiantes: Trujillo Sebastian y Sanoguera Agustin

Documentación del módulo servidor

Este documento explica cómo BACKEND/servidor.py inicia eShop Core, atiende las solicitudes del navegador y coordina el inventario, las reservas y las compras con SQLite. Describe el código vigente y permite seguir una operación desde la pantalla hasta su respuesta HTTP.

1 Función del servidor

El servidor es el punto de comunicación entre la interfaz web y la lógica del sistema. Entrega los archivos de FRONTEND, reconoce las rutas de la API, valida el formato recibido y llama al módulo que realiza cada operación. Las reglas de stock, FIFO y registro de compras se resuelven en los módulos de dominio y de persistencia.

Se utiliza la biblioteca estándar de Python. ThreadingHTTPServer atiende las solicitudes y ManejadorTienda, derivado de SimpleHTTPRequestHandler, define las respuestas. La dirección predeterminada es 127.0.0.1 y el puerto es 8000. La clase HTTPServer despacha las solicitudes a un manejador y ThreadingHTTPServer incorpora atención mediante hilos

2 Arranque del sistema

1. main.py importa iniciar_servidor y lo ejecuta cuando se inicia el programa. iniciar_tienda.bat es una alternativa para lanzar ese mismo punto de entrada.
2. iniciar_servidor calcula la ubicación de FRONTEND desde la ruta del propio archivo y llama a persistencia.inicializar_bd para preparar la base.
3. Carga una vez el inventario, las colas, la popularidad y la pila mediante cargar_inventario, cargar_colas, cargar_popularidad y cargar_pila.

4. Crea el diccionario de sesiones y un Lock. functools.partial proporciona esas mismas estructuras a cada instancia del manejador, junto con la carpeta pública.
5. ThreadingHTTPServer abre la dirección local y serve_forever mantiene la atención. Ctrl+C interrumpe el bucle; el bloque with cierra el servidor al salir.

3 Direcciones de las pantallas

La Tabla 1 resume direcciones de acceso a las pantallas.

Tabla 1

Direcciones de acceso a las pantallas

Ruta	Resultado
/	Respuesta 302 que redirige a /compra.html.
/compra.html	Portal de ingreso y catálogo del cliente.
/carrito.html	Productos elegidos, reservas, avisos y compras propias.
/index.html	Pantalla de gestión del inventario.

Estado compartido y comunicación

4 Componentes que recibe cada solicitud

El constructor de ManejadorTienda conserva referencias al inventario, las colas, la popularidad, la pila, las sesiones y el bloqueo. Cada solicitud tiene su propio manejador, pero trabaja con las mismas estructuras del proceso.

La Tabla 2 resume estado compartido entre solicitudes.

Tabla 2

Estado compartido entre solicitudes

Estado	Estructura y uso
inventario	TDA Lista con los productos cargados desde SQLite.
colas	Una instancia del TDA Cola por producto con clientes en espera.
popularidad	BST y conteos de unidades vendidas para las recomendaciones.
pila	TDA Pila con las compras confirmadas y la más reciente en la cima.
sesiones	Diccionario que relaciona token de ingreso con id_cliente.
bloqueo	Lock compartido para coordinar consultas y cambios sobre los TDA.

5 Recorrido general de una solicitud

1. JavaScript envía una solicitud con fetch. El método HTTP indica la acción y la ruta identifica el recurso.
2. do_GET, do_POST, do_PUT o do_DELETE obtiene la ruta con urlsplit. Los parámetros de recomendaciones se interpretan con parse_qs.
3. El manejador valida la solicitud y, cuando corresponde, identifica al cliente mediante X-Sesion. Bajo el bloqueo llama a la lógica de dominio.
4. La lógica consulta o modifica SQLite y los TDA. El resultado vuelve al manejador, que lo convierte a JSON mediante responder_json.
5. El navegador recibe el código HTTP y el resultado, actualiza los controles y muestra el mensaje correspondiente.

6 Entrega de archivos y respuestas JSON

Las solicitudes que no pertenecen a la API se delegan en SimpleHTTPRequestHandler, configurado para servir la carpeta FRONTEND. Las rutas de API desconocidas reciben 404 en formato JSON.

responder_json usa json.dumps con ensure_ascii=False y allow_nan=False, codifica el resultado en UTF-8 y envía Content-Type, Content-Length y Cache-Control: no-store. Esto conserva los caracteres del español y pide no reutilizar respuestas JSON almacenadas en caché. La conversión entre objetos de Python y JSON se apoya en el módulo

Referencia de rutas HTTP

Las rutas personales requieren un token válido en la cabecera X-Sesion. Las rutas actuales de inventario y recomendaciones no utilizan cliente_actual. La siguiente referencia documenta esa separación tal como está implementada.

7 Consultas con GET

La Tabla 3 resume rutas de consulta con GET.

Tabla 3

Rutas de consulta con GET

Ruta	Acción	Ingreso
/api/productos	Devuelve productos y agrega el conteo de popularidad a cada copia.	No
/api/clientes	Consulta los clientes registrados mediante persistencia.	No
/api/productos/ID/esperas	Consulta la cola del producto en orden FIFO. Devuelve [] si no hay esperas.	No
/api/recomendados?cantidad=5	Obtiene hasta la cantidad solicitada desde el BST. El límite admitido es de 1 a 50.	No
/api/sesion	Devuelve el cliente del token actual.	Sí
/api/mis-reservas	Devuelve solo las esperas del cliente, con producto y posición en su cola.	Sí
/api/mis-notificaciones	Consulta los avisos de reposición del cliente.	Sí
/api/mis-compras	Filtra el historial de la pila por el ID del cliente.	Sí

Desarrollo de Aplicacion con TAD

8 Operaciones que modifican datos

La Tabla 4 resume rutas que modifican datos.

Tabla 4

Rutas que modifican datos

Método y ruta	Datos recibidos y resultado
POST /api/ingreso	DNI; si es nuevo, nombre y apellido. Devuelve cliente y token o solicita completar el registro.
POST /api/salir	Objeto JSON vacío. Revoca el token de la sesión actual.
POST /api/esperas	Solo id_producto. Registra al cliente del token en la cola del producto.
POST /api/compras	Lista productos con id_producto y cantidad. Confirma la compra del cliente actual.
POST /api/productos	nombre, cantidad_disponible y precio. SQLite genera el ID.
PUT /api/productos/ID	Los tres campos del producto. Modifica datos y puede disparar la atención de una espera.
DELETE /api/productos/ID	El ID va en la ruta. Elimina el producto si no hay referencias que lo impidan.

Flujos de ingreso y reserva

9 Identificación por DNI

1. portal.js envía POST /api/ingreso con el DNI. responder_portal_post comprueba que las claves recibidas sean las esperadas.
2. persistencia.ingresar_cliente normaliza el documento y busca su registro. Si existe, devuelve el cliente. Si es nuevo y faltan sus datos, el servidor responde 404 con registrar=true.
3. El navegador habilita nombre y apellido y reenvía los datos. La persistencia valida y registra al cliente; el DNI es único.

4. El servidor crea un token con token_urlsafe(32), lo asocia al ID en sesiones y devuelve ambos. Las solicitudes siguientes incluyen X-Sesion.

cliente_actual resuelve el token y consulta que el cliente aún exista. Si el token es desconocido o el registro ya no existe, se responde 401. El servidor mantiene las sesiones solo en memoria: al reiniciarlo se debe ingresar nuevamente. El navegador conserva su token y el carrito en sessionStorage durante la pestaña.

10 Reserva de un producto sin stock

1. El cliente pulsa Reservar en el catálogo. El navegador envía POST /api/esperas con id_producto y su token; no elige manualmente un ID de cliente.
2. reservar_cliente_actual valida el formato y el ID. Bajo el Lock llama a cliente_actual y después a registrar_espera.
3. La persistencia verifica que el cliente y el producto existan, que el stock sea cero y que no haya una espera duplicada para ese par. Guarda la espera antes de modificar la cola.
4. arribo incorpora el registro al final del TDA Cola. El servidor devuelve 201 y el registro creado.

11 Consulta del turno del cliente

GET /api/mis-reservas recorre los productos y consulta cada cola con las primitivas del TDA. Enumera sus registros desde uno y selecciona los del cliente actual. La respuesta incluye nombre del producto, turno y datos de la espera. El recorrido conserva el orden final de la cola y se ejecuta bajo el mismo bloqueo.

Ejemplo de cuerpo enviado para reservar

```
{"id_producto": 3}
```

El registro persistente permite recuperar la espera al volver a ingresar con el mismo DNI, aunque se haya cerrado el navegador. Vaciar el carrito no elimina esa espera.

Flujos de compra e inventario

12 Confirmación de una compra

1. portal.js envía POST /api/compras con la lista de productos y cantidades. El cliente se obtiene del token, no del cuerpo de la solicitud.
2. responder_portal_post llama a registrar_transaccion bajo el bloqueo compartido. preparar_compra comprueba las cantidades, agrupa productos repetidos y verifica stock suficiente.
3. El backend toma los precios de su inventario y calcula el total. guardar_compra vuelve a verificar la disponibilidad y los precios dentro de una transacción SQLite.
4. En esa transacción se guardan la compra y sus detalles y se descuentan las unidades. Si falla, la base revierte los cambios de la operación.
5. Tras confirmar SQLite, se sincroniza la lista, se apila la compra y se actualiza el BST de popularidad sumando las unidades vendidas.
6. El servidor devuelve 201 con la compra. La pantalla vacía los productos del carrito y actualiza el catálogo y el historial personal.

Ejemplo de cuerpo enviado para comprar

```
{"productos": [{"id_producto": 1, "cantidad": 2}]}
```

13 Reposición y aviso al primero en espera

1. La pantalla de inventario envía PUT /api/productos/ID con nombre, cantidad_disponible y precio. El servidor verifica ruta, ID, JSON y campos.

2. modificar_producto compara el stock anterior con el nuevo. Cuando pasa de cero a positivo, consulta al primero de la cola y prepara el aviso.
3. SQLite confirma juntos el cambio del producto, la notificación persistente y la eliminación de la espera atendida. Luego la lista se actualiza y atencion retira al primero.
4. La respuesta contiene el producto y, si hubo atención, notificacion. El cliente consulta el aviso con GET /api/mis-notificaciones desde su carrito.

Se atiende a un cliente por transición de cero a positivo. La notificación es simulada: no envía correo, no reserva físicamente una unidad y no genera una compra. La compra posterior se realiza por separado.

14 Bajas y recomendaciones

DELETE /api/productos/ID llama a eliminar_producto. Primero se confirma la baja en SQLite y después se retira de la lista. Si una clave foránea impide borrar el registro, se devuelve 409. Para las recomendaciones, responder_recomendaciones valida cantidad y consulta el BST ordenado por unidades vendidas e ID.

Validación concurrencia y alcance

15 Controles de las solicitudes y códigos de respuesta

POST y PUT requieren Content-Type: application/json. El manejador exige un Content-Length entero mayor que cero y de hasta 65536 bytes, decodifica UTF-8 e interpreta JSON. Rechaza contenido mal formado y campos adicionales o faltantes. Los IDs de ruta deben ser enteros positivos; las validaciones de negocio se delegan a los módulos correspondientes.

La Tabla 5 resume códigos de respuesta HTTP.

Tabla 5

Códigos de respuesta HTTP

Código	Uso en el módulo
200	Consultas correctas, ingreso o salida, modificación y eliminación de producto.
201	Producto, espera o compra creados.
302	Redirección de la raíz al portal de compras.
400	Formato o datos inválidos y reglas de negocio incumplidas.
401	Token de sesión desconocido o cliente del ingreso inexistente.
404	Ruta inexistente, producto inexistente o DNI aún no registrado; este último habilita el registro.
409	La base impide eliminar un producto por referencias existentes.
415	El contenido enviado no es JSON.
500	Error SQLite controlado al consultar o confirmar una operación.

16 Coordinación de memoria y base de datos

ThreadingHTTPServer permite atender solicitudes concurrentes, pero las operaciones que usan estructuras compartidas entran en with self.bloqueo. El Lock evita que una consulta de cola que rota sus elementos se mezcle con una reserva o reposición. También protege la actualización del inventario y el recorrido temporal de la pila para consultar compras. El objeto Lock proporciona exclusión mutua para estas secciones de acceso compartido.

El bloqueo de Python y las transacciones SQLite cumplen funciones diferentes. El primero coordina los TDA del proceso; las segundas agrupan los cambios persistentes. Los módulos guardan antes de cambiar las estructuras. Al iniciar, las estructuras se reconstruyen desde SQLite.